Kevin Altomare

CS 250

Final Project – Sprint Review and Retrospective

October 2026

Applying Roles

As the Scrum Master for the SNHU Travel project, my main responsibility was keeping the team organized, communicating clearly, and making sure everyone understood what needed to happen during the sprint. Even though the course let me practice each role individually, in this retrospective I'm looking at the project from the Scrum Master's point of view and reflecting on how the team worked together.

The Product Owner played a big part in shaping the project. They talked with users, clarified traveler needs, and updated acceptance criteria when something changed. A good example was the Detox and Wellness destination story. The Product Owner realized the story needed to head in a new direction, so they updated the acceptance criteria and explained the new direction. I made sure the team understood those changes and that the updated story was ready for the next increment.

The Developer depended on clear acceptance criteria and quick feedback. Whenever something changed, the Developer needed to know exactly what was different so they didn't waste time building the wrong thing. During Daily Scrums, I encouraged the Developer to bring up blockers early so the Product Owner could answer questions right away.

The Tester helped the team catch gaps in the user stories. Early on, the Tester noticed that the stories didn't explain how the destination list would actually look. Once the Product Owner clarified that it would be a slideshow, the Tester had to revise all three test cases. I helped facilitate those conversations so the team could adjust quickly instead of getting stuck.

Overall, my role was to keep communication open, keep the Scrum events consistent, and make sure the team felt comfortable speaking up. When everyone understood their responsibilities and had a chance to collaborate, the sprint ran much smoother.

Completing User Stories

Agile made it easier for the team to complete user stories because the process expects change and encourages constant communication. User stories gave us a clear picture of who the traveler was, what they wanted to do, and why it mattered. That helped the team stay focused on real user needs instead of building features just because they sounded interesting.

During Sprint Planning, the Product Owner walked us through the highest priority items, and the team broke them down into smaller tasks. This gave everyone a shared direction and made it clear what the sprint was supposed to accomplish.

A good example of how Agile helped user stories reach completion was the Detox and Wellness destination story. Originally, the feature was written for general travel destinations, but the Product Owner updated the requirements, so the list would focus specifically on detox and wellness options. This change meant the acceptance criteria had to be rewritten, the Tester had to revise the test cases, and the Developer had to adjust the implementation to match the new traveler expectations. Daily Scrums and backlog refinement sessions made it easy for the team to talk through the update, clarify what the new version of the feature needed to show, and adjust the work without delaying the sprint. Because Agile supports quick changes, the team was able to shift direction and still complete the story on time.

The same thing happened with the test cases. When the Tester realized the interface details were missing, we were able to revise the acceptance criteria and test cases within the

same sprint. Agile's flexibility made it possible to finish the story accurately instead of pushing it to a later release.

Handling Interruptions

Interruptions are normal in software development, and Agile made them much easier to handle. Midway through the project, the Product Owner updated the Detox and Wellness story based on new traveler feedback. Instead of stopping everything, we adjusted the backlog, refined the acceptance criteria, and planned the updated work for the next increment.

Another interruption happened when the Tester had already created detailed test cases but had to revise them after the Product Owner clarified the design. In a Waterfall environment, this would have caused delays because the team would have had to reopen earlier phases. Under Agile, we simply updated the backlog, revised the test cases, and kept moving.

From the Developer's perspective, Agile made these interruptions manageable because the process expects change. User stories can be refined, priorities can shift, and estimates can be revisited without derailing the whole project. As Scrum Master, I helped the team stay calm and focused by keeping communication open and making sure everyone understood the new direction.

Communication

Communication was one of the biggest reasons the SNHU Travel project stayed on track. Agile often depends on transparency, and the course made it clear how important it is to keep information visible and easy to access.

Information radiators, like sprint boards and backlogs, helped the team see progress at a glance. When everyone could see what was in progress, what was blocked, and what was finished, it removed confusion and kept the team aligned. Daily Scrums also helped because they gave each team member a chance to share what they were working on and what they needed help with. These short check-ins prevented problems from sitting unnoticed.

Communication between roles mattered just as much. The Tester asked the Product Owner clarifying questions whenever user stories were vague. The Developer communicated blockers clearly and explained why certain information was needed. As Scrum Master, I encouraged open dialogue and kept meetings focused so the team could surface issues early.

One of the clearest examples of effective communication came from the Module 6 group discussion. I sent a message to the team summarizing our chosen Agile practices and making sure everyone agreed on how we would support Vision Quest's transition to Agile. My message said:

"Hey team,
Even though a few of us ended up choosing the same roles, we still pulled together a solid set of Agile practices that match what Vision Quest needs. To meet the requirement of implementing at least one practice from each Scrum role, here's the set I propose agreeing to move forward with:
Developer: Cross-functional collaboration

Tester: Continuous testing

Scrum Master: Daily Scrum

Product Owner: Backlog prioritization using user stories

These four practices cover the major gaps described in the case study. Cross-functional collaboration helps fix the issue of isolated roles. Continuous testing keeps quality high and prevents last-minute surprises. Daily Scrum keeps everyone aligned and makes blockers visible early. Backlog prioritization ensures that we're always working on the most valuable items first. We can still use other practices like Sprints, Definition of Done, and Retrospectives as we go, but the four above represent the core recommendations from each role. This gives us a complete and balanced Agile approach for Vision Quest.

Thanks everyone for your input,

Kevin"

Including this kind of communication helped the team stay aligned and made sure everyone understood how their role contributed to the project. It also showed how simple, direct communication can keep an Agile team organized and moving forward. That same style of communication helped us throughout the SNHU Travel project.

Organizational Tools

Organizational tools played a huge role in keeping the team aligned. Tools like Azure Boards and JIRA gave us a single place to track tasks, user stories, bugs, and sprint progress. These tools made everything visible, which is important in an Agile environment where priorities can shift quickly.

Scrum events also helped keep the project organized. Sprint Planning gave the team direction, Daily Scrums kept everyone aligned, Backlog Refinement made sure items were clear and ready, Sprint Reviews kept the work grounded in real customer value, and Retrospectives

helped the team improve its process. Together, these events created a rhythm that kept the team from feeling overwhelmed.

Acceptance criteria, test cases, story maps, and user stories also helped the team stay focused on traveler needs. These artifacts made expectations clear and prevented misunderstandings that could lead to rework.

Evaluating Agile Process

The Scrum Agile approach worked well for the SNHU Travel project. One of the biggest advantages was how easy it was to adapt to changing requirements. When the Detox and Wellness story changed direction, Agile allowed us to update the backlog and adjust the work without delaying the project. Continuous communication and iterative development also helped the team catch issues early, like missing interface details in the test cases.

Agile also supported cross-functional collaboration. Because each role communicated often and worked together, the team was able to refine user stories, clarify acceptance criteria, and deliver working increments consistently. Tools like Azure Boards and daily Scrums helped maintain transparency and alignment.

There were challenges, too. Agile requires constant communication, and changing requirements sometimes created extra work for the Tester and Developer. The team also had to stay focused during sprints to avoid getting distracted by new ideas.

Even with those challenges, Agile was the best approach for the SNHU Travel project. Traveler needs can change quickly, and the project required flexibility, collaboration, and fast iteration. Agile supported all of that by helping the team respond to changes, refine user stories,

and deliver working software consistently. Being able to adapt mid sprint was a major reason the project stayed on track.

References

Schwaber, K., & Sutherland, J. (2020). The Scrum guide: The definitive guide to Scrum. Scrum.org. https://scrumguides.org/scrum-guide.html

GeeksforGeeks. (2026, April 30). Agile vs waterfall model in software development. https://www.geeksforgeeks.org/software-engineering/agile-vs-waterfall/

Usmani, F. (2026, April 24). Agile team roles & responsibilities. PM Study Circle. https://pmstudycircle.com/agile-team-roles/

GeeksforGeeks. (2026, July 7). Waterfall model—Software engineering. https://www.geeksforgeeks.org/software-engineering/waterfall-model/

Zhezherau, A. (2026, July 13). The agile software development life cycle. Wrike. https://www.wrike.com/agile-guide/agile-development-life-cycle/